

Smart Document Scanner and Enhancer using Deep Learning

Arin Singh Mashta

Vishesh Bhargav

Jaya Meena

Summary :

The growing number of digital documents requires an automated system for categorization and retrieval. This project introduces a deep learning system that flattens and enhances scanned documents. It uses image-based analysis to improve document organization and retrieval. The system applies convolutional neural networks (CNNs) for image classification and transformer-based model for processing, enabling accurate function even in complex cases.

Objectives :

Detecting four corners of a page in an unconstrained photo using a CNN with a contour-based fallback for edge cases.

Implement OCR and transformer models to analyse text extracted from documents for enhanced classification.

We try to enhance readability through a DL model for a cleaner with adaptive contrast stretching of document page.

Create a user-friendly application for seamless document scanning, classification, and retrieval.

Future Goals:

Develop a CNN-based document classification model to categorize scanned documents (e.g., invoices, receipts, reports).

Integrate multimodal fusion techniques to improve accuracy using both image and text-based features.

Build an automated tagging system to assign relevant keywords based on document content.

Methodology :

1. Data Collection

- Gather a dataset of scanned document images from sources mentioned in Dataset part
- Capture images under different lighting conditions and angles to improve model performance.

2. Preprocessing

- Image Preprocessing: Resize images, convert to grayscale, and apply noise reduction techniques.

- Document Edge Detection: Use U-Net or Mask R-CNN to detect document boundaries and extract regions of interest.
- Perspective Transformation: Apply homography transformation to align and straighten documents.

3. System Integration

- Develop a mobile/desktop application for scanning, categorizing, and retrieving documents.
- Implement real-time processing for document classification and tagging upon upload.

Data Extraction & Modelling:

Document Data Extraction:

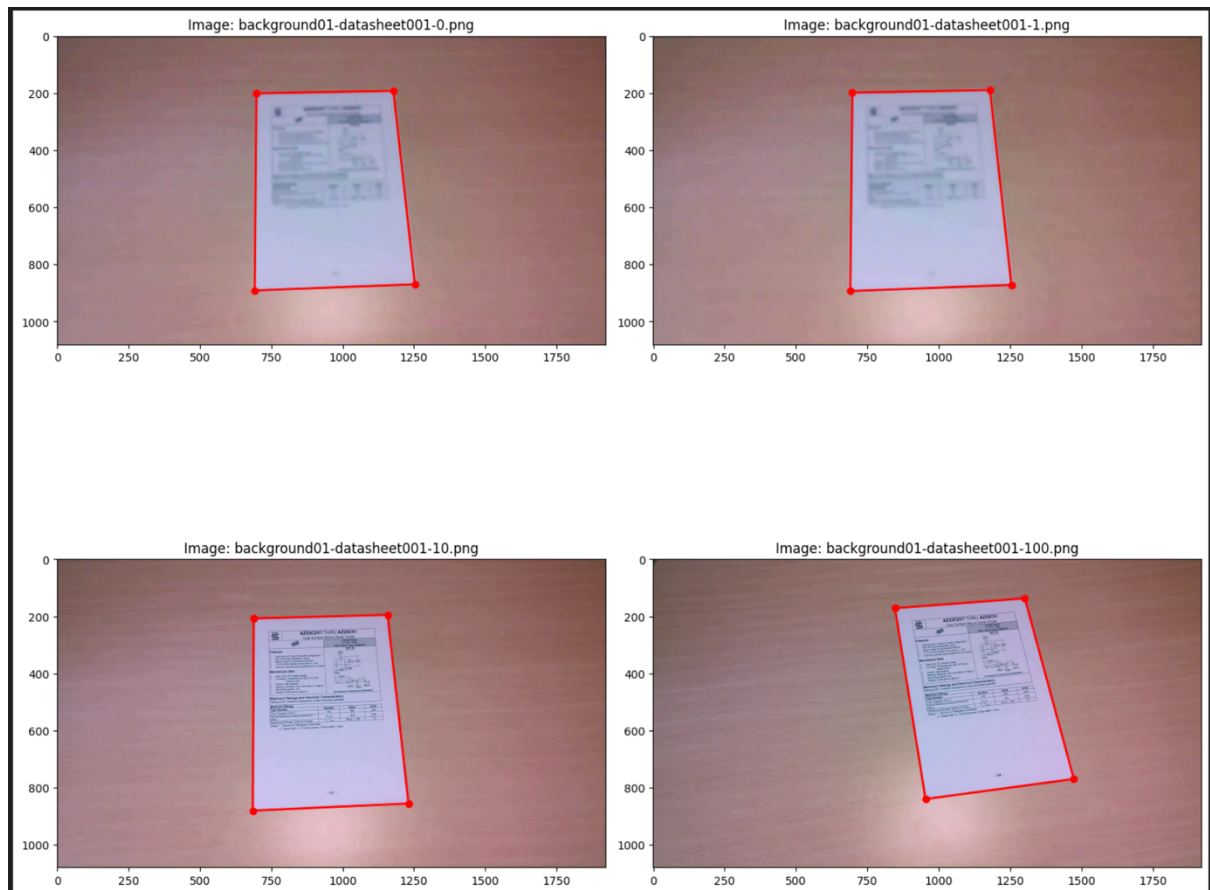
We import necessary libraries such as numpy and pandas which are essential for basic data preprocessing and analysis step.

Now, our code starts by reading the CSV file `frame_data.csv` into a pandas DataFrame called `df`. Then it uses a pivot operation to reorganize the data such that each `frame_filename` becomes a row, and each name splits into two columns for x and y. After pivoting, it flattens the resulting multi-level column indices into a single level by combining coordinate and corner into a single string. It then resets the index and renames `frame_filename` to `image_name` for clarity. Finally, the script exports the transformed DataFrame back to a new CSV file called `updated_frame_data.csv` and prints the first few rows to verify the changes.

	image_name	x_bl	x_br	x_tl	x_tr	\
0	background01-datasheet001-0.png	692.141	1253.18	698.087	1178.15	
1	background01-datasheet001-1.png	690.063	1254.61	697.230	1179.66	
2	background01-datasheet001-10.png	685.454	1230.92	688.902	1157.41	
3	background01-datasheet001-100.png	953.840	1473.28	846.754	1301.69	
4	background01-datasheet001-101.png	956.263	1474.31	847.800	1301.88	
	y_bl	y_br	y_tl	y_tr		
0	891.077	869.656	200.476	191.515		
1	893.292	871.689	197.947	188.863		
2	880.838	855.719	207.181	194.919		
3	840.161	769.773	171.333	137.063		
4	838.829	769.172	173.033	138.885		

In the next step we import libraries for plotting and image processing, then set up a figure with a 2x2 grid of subplots. Our code loops through rows of a DataFrame, up to a specified limit, and constructs a path to each image. We read each image using `cv2`, converts the color space from BGR to RGB, and displays the image in a subplot. We also try to extract corner coordinates from the DataFrame to form a bounding box, then plots these points and connects them with a line.

Finally, it assigns a title displaying the image name for each subplot, adjusts the layout for a clean display, and shows the resulting plot.



Next step of our code defines a custom DocumentDataset class that retrieves images and bounding box labels from a DataFrame. Inside the getitem method, it constructs the image path, opens the file as an RGB image, and handles missing files by returning None.

If a transform is provided, such as resizing or normalization, it applies it to the image. After loading, the script gathers the corner coordinates from the DataFrame row and converts them into a tensor for the label. A custom collate_fn function filters out any samples where the image is missing, then stacks the valid images and labels into tensors for efficient batch processing.

Finally, we use train_test_split to divide the DataFrame into training, validation, and test sets, and creates DataLoaders that manage batching, shuffling, and application of collate_fn. The result is a well-structured pipeline for loading, transforming, and batching images and labels for deep learning work.

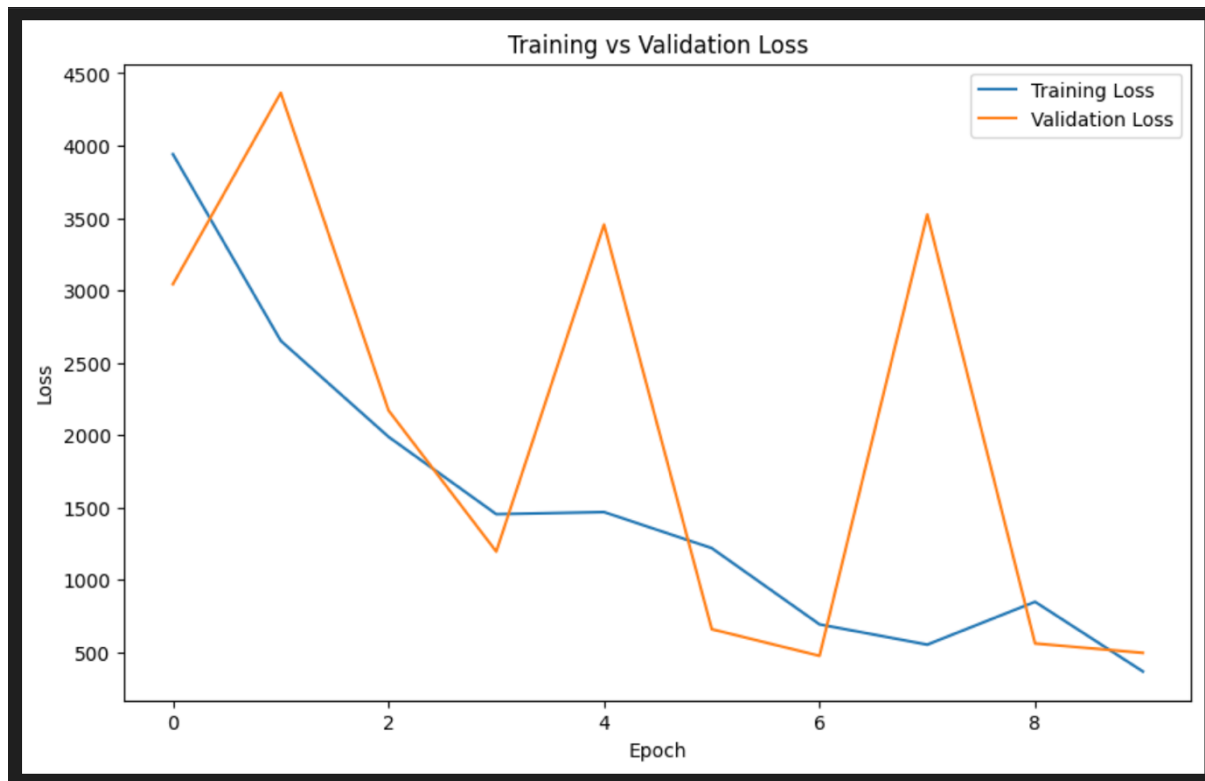
We in the next step define and train a convolutional neural network to predict document corner coordinates from input images. The model, DocumentCornerModel, consists of stacked convolutional and pooling layers for feature extraction, followed by fully connected layers that output 8 floating-point values representing the x and y positions of four corners.

It uses mean squared error (MSE) loss for regression and the Adam optimizer with a learning rate of 0.001. The training loop runs for a fixed number of epochs, where each epoch processes the training data in batches, computes predictions, evaluates the loss, and updates model weights through backpropagation.

After each epoch, it also evaluates performance on the validation dataset without updating weights, storing both training and validation losses. Invalid batches with missing images or labels are skipped during both training and validation. Finally, the training and validation losses across epochs are printed for performance monitoring.

```
Epoch 1/10: 100%|████████████████████| 467/467 [14:07<00:00, 1.81s/it]
Epoch 1, Train Loss: 3940.2831
Validation Epoch 1/10: 100%|████████████████████| 156/156 [04:10<00:00, 1.61s/it]
Epoch 1, Validation Loss: 3043.3290
Epoch 2/10: 100%|████████████████████| 467/467 [14:45<00:00, 1.90s/it]
Epoch 2, Train Loss: 2652.4421
Validation Epoch 2/10: 100%|████████████████████| 156/156 [04:20<00:00, 1.67s/it]
Epoch 2, Validation Loss: 4364.2929
Epoch 3/10: 100%|████████████████████| 467/467 [14:58<00:00, 1.92s/it]
Epoch 3, Train Loss: 1988.8424
Validation Epoch 3/10: 100%|████████████████████| 156/156 [04:16<00:00, 1.65s/it]
Epoch 3, Validation Loss: 2171.4857
Epoch 4/10: 100%|████████████████████| 467/467 [15:18<00:00, 1.97s/it]
Epoch 4, Train Loss: 1454.5741
Validation Epoch 4/10: 100%|████████████████████| 156/156 [04:20<00:00, 1.67s/it]
Epoch 4, Validation Loss: 1196.2107
Epoch 5/10: 100%|████████████████████| 467/467 [14:58<00:00, 1.92s/it]
Epoch 5, Train Loss: 1469.0772
Validation Epoch 5/10: 100%|████████████████████| 156/156 [04:16<00:00, 1.65s/it]
Epoch 5, Validation Loss: 3454.4549
Epoch 6/10: 100%|████████████████████| 467/467 [15:25<00:00, 1.98s/it]
Epoch 6, Train Loss: 1219.8042
Validation Epoch 6/10: 100%|████████████████████| 156/156 [04:15<00:00, 1.64s/it]
Epoch 6, Validation Loss: 659.8950
Epoch 7/10: 100%|████████████████████| 467/467 [15:02<00:00, 1.93s/it]
Epoch 7, Train Loss: 693.1800
Validation Epoch 7/10: 100%|████████████████████| 156/156 [04:28<00:00, 1.72s/it]
Epoch 7, Validation Loss: 476.0022
Epoch 8/10: 100%|████████████████████| 467/467 [14:54<00:00, 1.92s/it]
Epoch 8, Train Loss: 553.5178
Validation Epoch 8/10: 100%|████████████████████| 156/156 [04:16<00:00, 1.64s/it]
Epoch 8, Validation Loss: 3524.4663
Epoch 9/10: 100%|████████████████████| 467/467 [27:44<00:00, 3.56s/it]
Epoch 9, Train Loss: 849.1151
Validation Epoch 9/10: 100%|████████████████████| 156/156 [06:09<00:00, 2.37s/it]
Epoch 9, Validation Loss: 561.5994
Epoch 10/10: 100%|████████████████████| 467/467 [23:03<00:00, 2.96s/it]
Epoch 10, Train Loss: 368.2644
Validation Epoch 10/10: 100%|████████████████████| 156/156 [06:26<00:00, 2.48s/it]
Epoch 10, Validation Loss: 496.3247
```

```
Training Losses: [3940.2830598818923, 2652.442144422511, 1988.8423844854121, 1454.574074853411, 1469.077245218085, 1219.8042488833312, 693.1800299895652, 553.5178245528097, 849.1151198523969, 368.2644324945944]
Validation Losses: [3043.3289677546572, 4364.292903019832, 2171.4856770833335, 1196.2107150738057, 3454.4548785869893, 659.8949933174329, 476.00220156938605, 3524.466322678786, 561.5993556487255, 496.32474341759314]
```

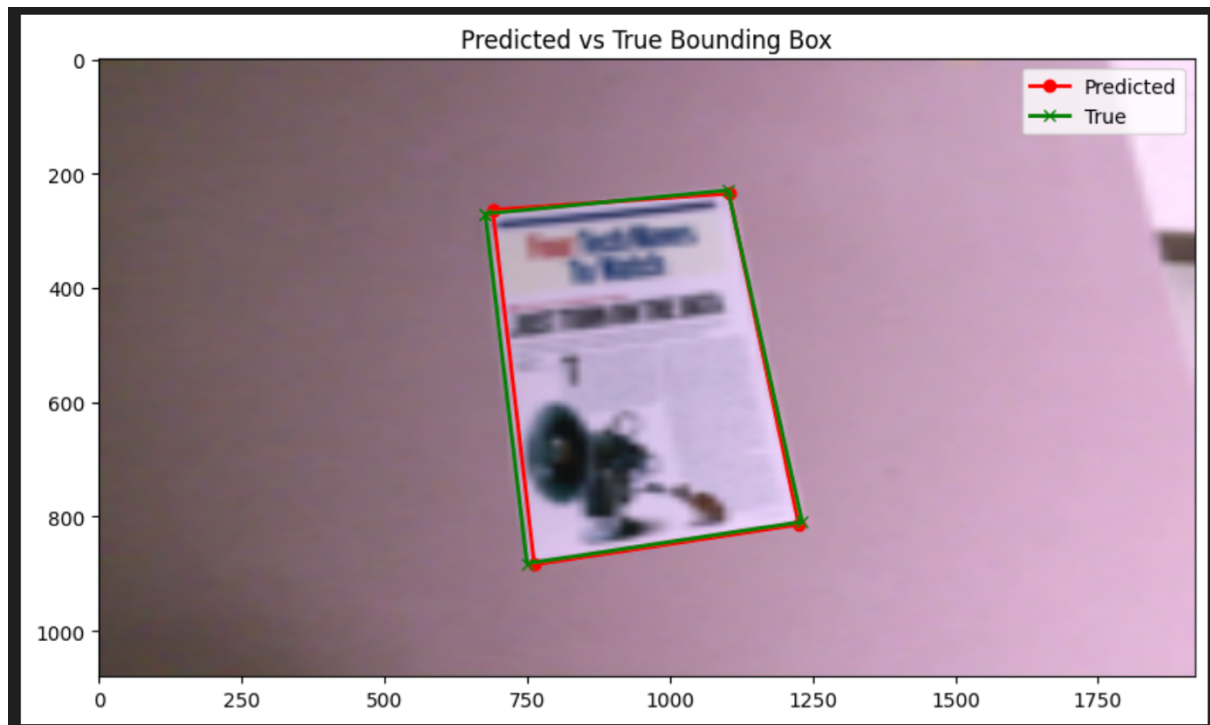


This plot displays the training and validation loss on ten epochs. The training loss shows a consistent downward trend, indicating that the model is learning effectively over time. However, the validation loss exhibits high fluctuations, spiking dramatically at certain epochs, particularly at epochs 1, 4, and 7. This inconsistency suggests that the model is trying to generalize and learning its weights. Although the training loss improves steadily. Overall, while training performance is improving, model is working fine and can be tuned further for smooth graph.

We now check model on our test dataset using the R^2 metric. It begins by loading the model's saved weights and setting the model to evaluation mode. It then iterates through the test data using a DataLoader, ensuring that no gradients are computed to save memory and computation. For each batch, it checks if the images or labels are None and skips them if so. After processing all batches, it calculates the R^2 score to quantify how well the model's predictions match the true values, printing the result to four decimal places.

```
Validation R2: 0.9839
```

At end we visualize and compare the predicted and actual coordinates of bounding boxes on an image from a test dataset. It first retrieves the first test image, converts it from a PyTorch tensor to a NumPy array, and then rescales it to a standard 1080p resolution using OpenCV. Then, it extracts the predicted and true bounding box coordinates for that image, rearranges them into a specific order, and casts them to integers for plotting. Using Matplotlib, it displays the image and overlays the predicted bounding box in red and the true bounding box in green, both connected in closed loops with labelled points. This provides a visual assessment of the model's localization accuracy.



Integrating various Deep Learning models and compiling it towards our final objectives:

In our document corner detection and flattening module we try implementing DocumentCornerModel which is a convolutional neural network designed to regress the coordinates of a document's four corners within an input photograph.

Our deep learning network accepts three-channel RGB images and processes them through four convolutional blocks, each consisting of a 3×3 convolution, ReLU activation, and a 2×2 max-pool. These blocks progressively grow the channel depth ($3 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 256$) while quartering the spatial resolution at every stage, so a 128×128 input would emerge from the final block as a compact 256-channel feature map of size 8×8 .

After flattening we pass the features through a three-layer fully connected head which are a dense layer mapping $256 \times 8 \times 8 = 16,384$ features to 512, which is a ReLU, another dense layer to 256, a second ReLU, and a final linear layer that outputs eight real values. These eight numbers are intended to encode the pixel coordinates of the top-left, top-right, bottom-right, and bottom-left corners, respectively, which enables subsequent perspective correction or document cropping downstream.

In our image enhancement deep learning module, we define and implement a lightweight PyTorch model of the U-Net architecture tailored for single-channel (grayscale) image-to-image tasks such as document binarization. Inside the Unet class, we define a small helper function convblock constructs two successive 3×3 convolution layers, each followed by batch-normalization and an in-place ReLU which gives every block the capacity to learn rich local features while keeping gradients stable.

The encoder path is built with the help of three blocks (enc1–enc3) that progressively expand the feature dimension from $1 \rightarrow 64 \rightarrow 128 \rightarrow 256$ while spatially down-sample through a shared 2×2 max-pool. After the deepest encoder stage, a bottleneck doubles the channels to 512, providing a compact global representation.

The decoder mirrors the encoder which means three transposed-convolution up-sampling stages (up3–up1) halve the channel count at each step and are concatenated with their encoder counterparts before passing through symmetric decoder blocks (dec3–dec1). Finally, a 1×1 convolution reduces the features back to a single channel and torch.sigmoid squashes the output to the $[0, 1]$ range, making it suitable for pixel-wise masks or enhanced grayscale images.

In our module we use the FastAPI in the backend when the clients hit, they want to turn a raw photograph of a document into a flattened, contrast-enhanced scan. We first initiate a FastAPI app titled Doc Scanner and build a small folder hierarchy beside the project root, a frontend directory for static assets, an uploads folder that temporarily stores the original user images, and an outputs folder that holds the processed results.

Each directory is created at launch if it is missing. The app then mounts two static routes: /static exposes everything inside the frontend folder so a browser can fetch the single-page interface, and /files exposes completed scans so they can be downloaded by URL. The root GET endpoint simply streams back index.html, allowing the same Python service to deliver both the UI and the inference API.

In the POST /process endpoint, when a user uploads a picture, the handler first rejects anything that is not a PNG or JPEG to avoid incompatible formats. It gives every upload a UUID prefix, saves the raw bytes to a disk, and prints a debug statement so failures can be traced later. Next, we try opening the image with Pillow and pass it to us detect_and_flatten() to deskew and crop the page and feeds the flattened result to enhance() for denoising. Both helper functions live in pipeline and hide the deep-learning details. If either step fails an exception is caught, the full traceback is printed, and the user receives a 500 error with the message string. Otherwise, the enhanced scan is written to outputs/ with the suffix _enhanced.png, and the endpoint returns a JSON object containing the public URL so the frontend can display the result.

Understanding output of our project:

Step1: Original Input Image



Step2: Document box bounding and flattening



Step3: Enhancing image and final result



Step4: Comparing with our benchmark: Cam Scanner



Final Deployed Model and Its results:

Landing Page:

User will land on this page and can upload any image of their choice in any of the formats, such as png, jpeg, jpg from their local system files and click on process images button to get the final output.



Document Scanner

Snap • Fix • Save

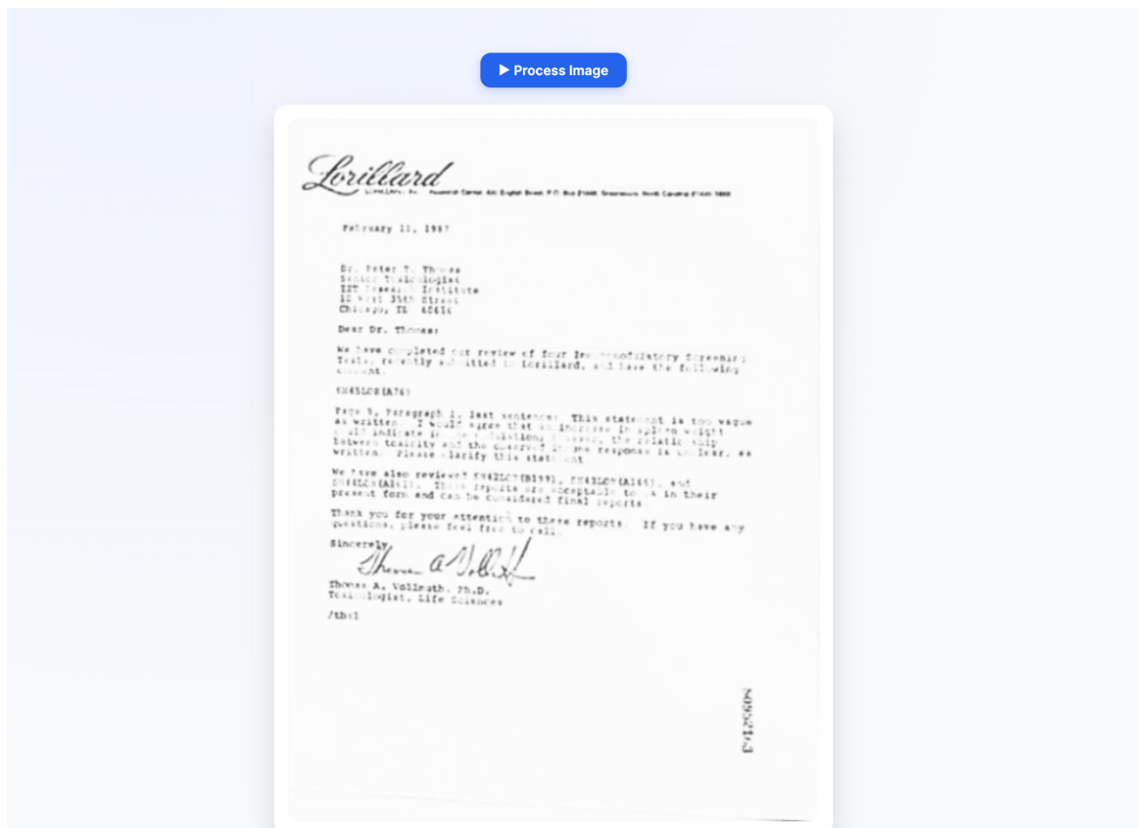
👉 Drag an image here or click to browse

📁 Choose Image

✅ Image uploaded successfully

▶ Process Image

After pressing the process images button, we get the processed image as output on our website.



Future Goals:

These are some extra features which need more complex collection of data and much more computation power to achieve the desired results. But we have tried some feature implementation and tried observing their results.

Text Extraction:

The Financial Documents clustering data consists of 5 predefined tables: income statements, balance sheets, cash flows, notes and others of public companies which we will be using for training our text extraction task

Code pipeline:

The `html_to_img_converter` transforms HTML documents into JPG images. It uses `imgkit` to render the HTML content. The function resizes images based on a scaling factor. Output images are saved in a specified directory.

```
reader = easyocr.Reader(['en'])
```

The `get_bbox_coordinates` function processes EasyOCR's detected text regions by extracting the minimum and maximum x/y coordinates from the bounding polygon to form a rectangular box, which is then drawn on the original image which is on the left side while the corresponding extracted text is displayed in its spatial position on the right side, `classification_model = LayoutLMv3ForSequenceClassification.from_pretrained ("microsoft/layoutlmv3-base", num_labels=2)`

[illegible]

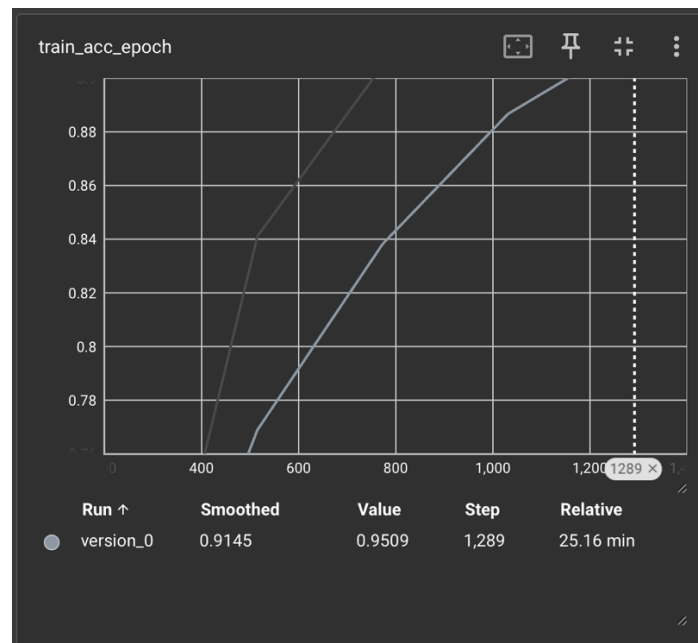
A	Description	12/31/2016		12/31/2015	
		On Cr	On Dr	On Cr	On Dr
1	Assets				
11	Net Current Assets				
111	Property, Plant and Equipment	50,780.00	5,808.89	50,780.00	5,808.89
112	Intangible Assets	500.00	57.75	500.00	57.75
113	Financial Assets	500.00	500.00	50.00	50.00
114	Other	500.00	500.00	50.00	50.00
115	Other Financial Assets	500.00	500.00	50.00	50.00
116	Other Non-current Assets	-	-	-	-
117	Other Non-current Assets	-	-	-	-
118	Other Non-current Assets	-	-	-	-
119	Other Non-current Assets	-	-	-	-
12	Current Assets	5,000.00	5,000.00	5,000.00	5,000.00
121	Financial Assets	5,000.00	5,000.00	5,000.00	5,000.00
122	Financial Assets	5,000.00	5,000.00	5,000.00	5,000.00
123	Financial Assets	5,000.00	5,000.00	5,000.00	5,000.00
124	Financial Assets	5,000.00	5,000.00	5,000.00	5,000.00
125	Financial Assets	5,000.00	5,000.00	5,000.00	5,000.00
126	Financial Assets	5,000.00	5,000.00	5,000.00	5,000.00
127	Financial Assets	5,000.00	5,000.00	5,000.00	5,000.00
128	Financial Assets	5,000.00	5,000.00	5,000.00	5,000.00
129	Financial Assets	5,000.00	5,000.00	5,000.00	5,000.00
13	Other Assets	5,000.00	5,000.00	5,000.00	5,000.00
131	Other Assets	5,000.00	5,000.00	5,000.00	5,000.00
132	Other Assets	5,000.00	5,000.00	5,000.00	5,000.00
133	Other Assets	5,000.00	5,000.00	5,000.00	5,000.00
134	Other Assets	5,000.00	5,000.00	5,000.00	5,000.00
135	Other Assets	5,000.00	5,000.00	5,000.00	5,000.00
136	Other Assets	5,000.00	5,000.00	5,000.00	5,000.00
137	Other Assets	5,000.00	5,000.00	5,000.00	5,000.00
138	Other Assets	5,000.00	5,000.00	5,000.00	5,000.00
139	Other Assets	5,000.00	5,000.00	5,000.00	5,000.00
14	Other Assets	5,000.00	5,000.00	5,000.00	5,000.00
141	Other Assets	5,000.00	5,000.00	5,000.00	5,000.00
142	Other Assets	5,000.00	5,000.00	5,000.00	5,000.00
143	Other Assets	5,000.00	5,000.00	5,000.00	5,000.00
144	Other Assets	5,000.00	5,000.00	5,000.00	5,000.00
145	Other Assets	5,000.00	5,000.00	5,000.00	5,000.00
146	Other Assets	5,000.00	5,000.00	5,000.00	5,000.00
147	Other Assets	5,000.00	5,000.00	5,000.00	5,000.00
148	Other Assets	5,000.00	5,000.00	5,000.00	5,000.00
149	Other Assets	5,000.00	5,000.00	5,000.00	5,000.00
15	Other Assets	5,000.00	5,000.00	5,000.00	5,000.00
151	Other Assets	5,000.00	5,000.00	5,000.00	5,000.00
152	Other Assets	5,000.00	5,000.00	5,000.00	5,000.00
153	Other Assets	5,000.00	5,000.00	5,000.00	5,000.00
154	Other Assets	5,000.00	5,000.00	5,000.00	5,000.00
155	Other Assets	5,000.00	5,000.00	5,000.00	5,000.00
156	Other Assets	5,000.00	5,000.00	5,000.00	5,000.00
157	Other Assets	5,000.00	5,000.00	5,000.00	5,000.00
158	Other Assets	5,000.00	5,000.00	5,000.00	5,000.00
159	Other Assets	5,000.00	5,000.00	5,000.00	5,000.00
16	Other Assets	5,000.00	5,000.00	5,000.00	5,000.00
161	Other Assets	5,000.00	5,000.00		

The LayoutLMv3Processor prepares inputs for LayoutLMv3. It processes text, bounding boxes, and images. The processor handles OCR data, tokenization, and image normalization.

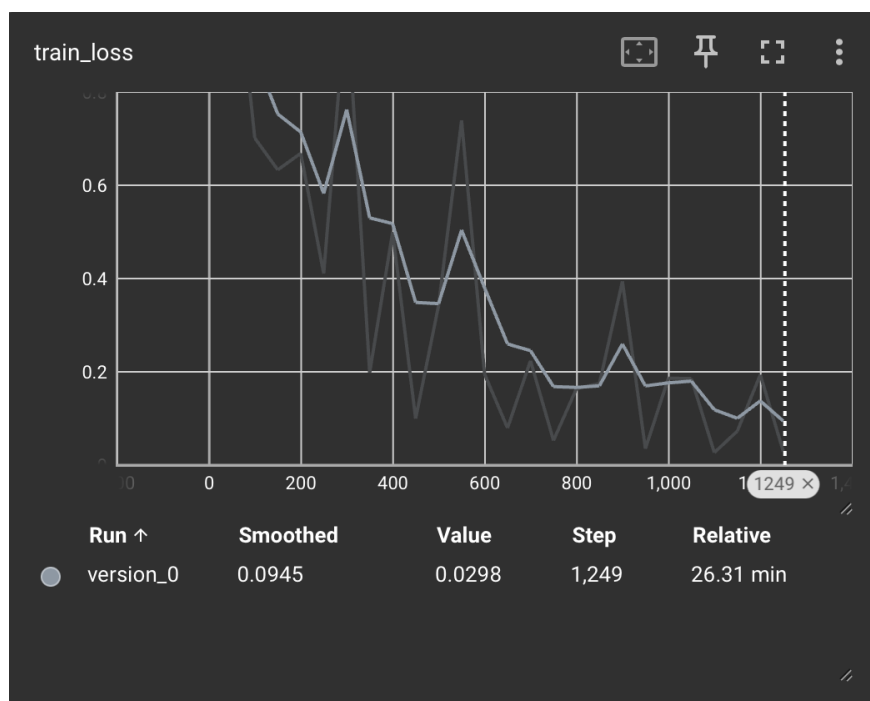
The `training_step` and `validation_step` methods define the training and validation steps respectively. In each method, the input tensors are passed through the model, and the loss and accuracy are logged. The `configure_optimizers` method defines an Adam optimizer used for training.

We train our model for 5 epochs and achieve the following results

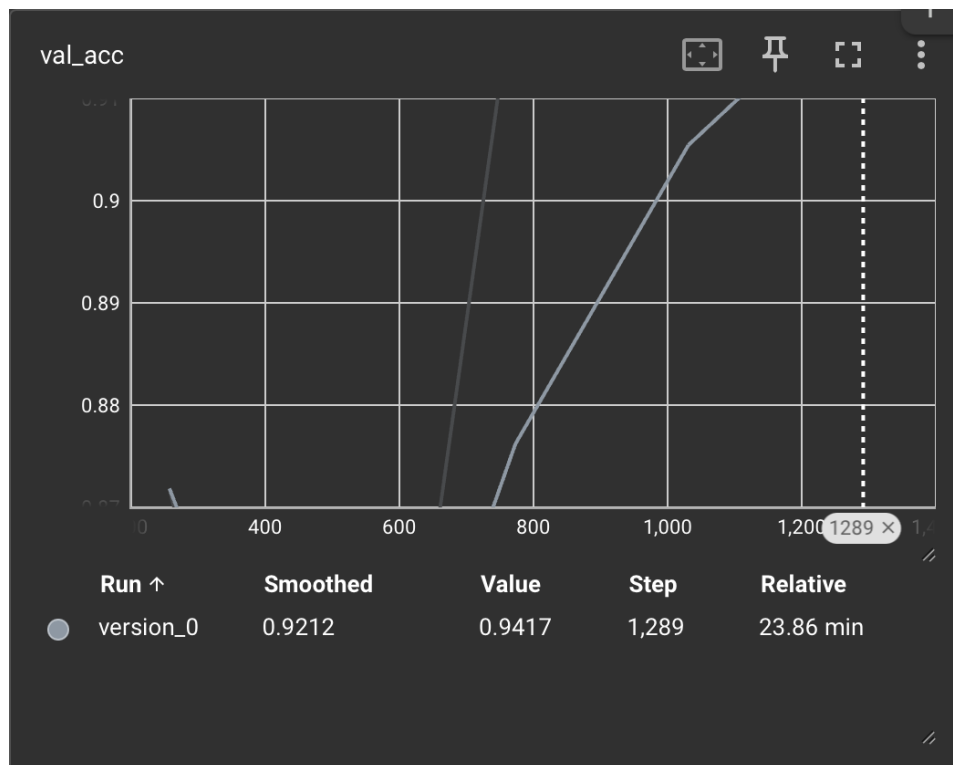
We achieve a train accuracy of 95.09



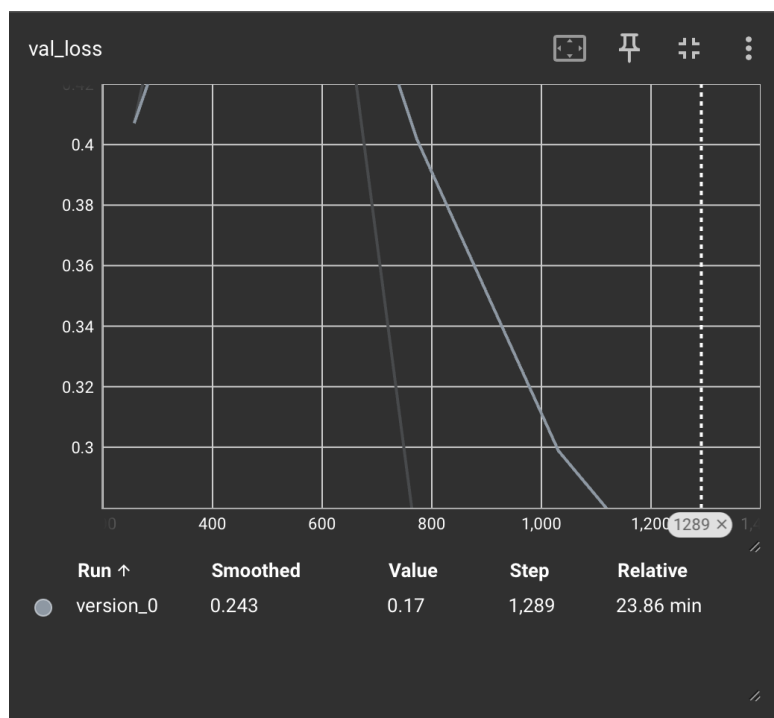
We have a train loss of 0.029



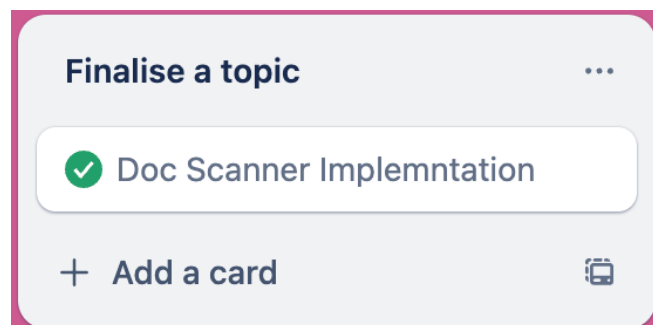
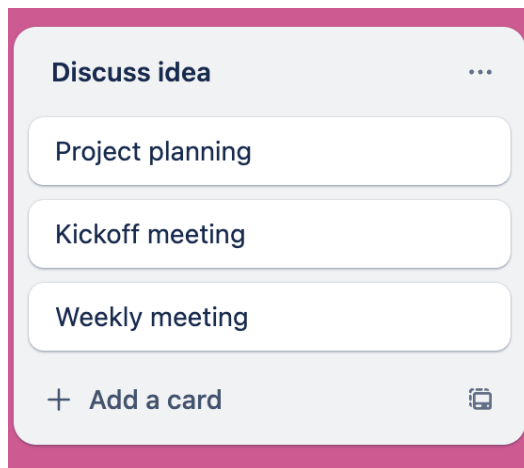
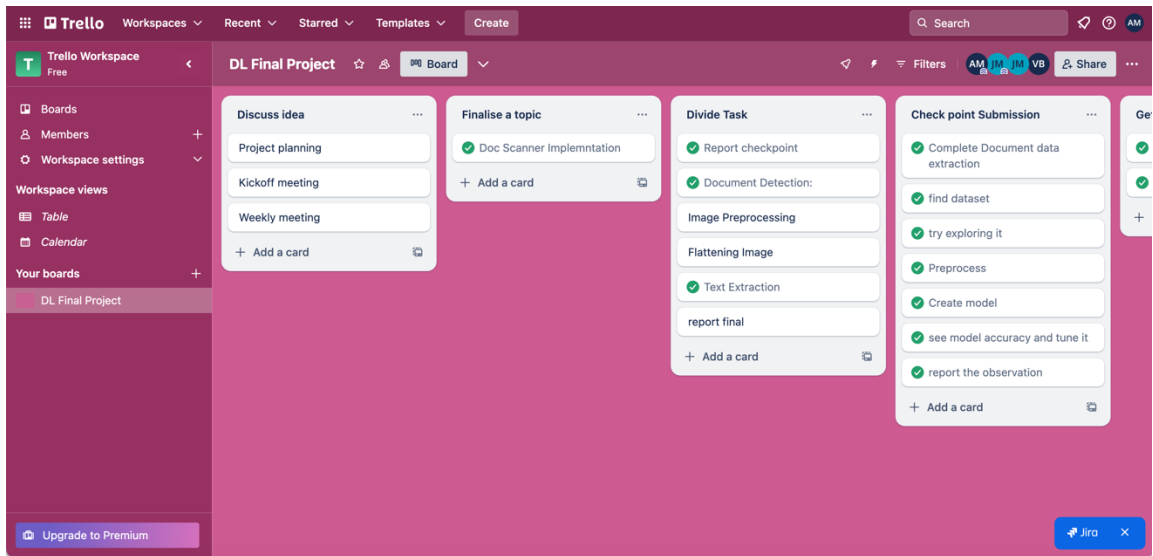
We achieve a validation accuracy of 94.17



We have a validation loss of 0.17



Project Management Tool: Trello (Till Checkpoint)



Get Dataset ...

✓ Get dataset for Text extraction

✓ get dataset for doc extraction

+ Add a card 

Divide Task ...

✓ Report checkpoint

✓ Document Detection:

Image Preprocessing

Flattening Image

✓ Text Extraction

report final

+ Add a card 

Check point Submission ...

✓ Complete Document data extraction

✓ find dataset

✓ try exploring it

✓ Preprocess

✓ Create model

✓ see model accuracy and tune it

✓ report the observation

+ Add a card 

SCRUM Meetings

✓ Attended 1st meeting with Rohan Sharma in OH

+ Add a card

Weekly update meetings

✓ Meeting on 8th March

JM VB AM

✓ Meeting on 15th March

VB JM AM

✓ Team meet on 24 March

JM VB AM

✓ Meet on 1st April

AM JM VB

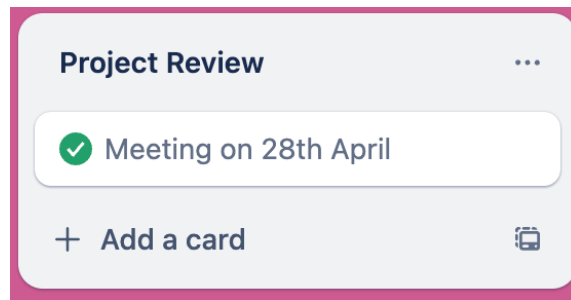
+ Add a card

Brainstorming Meetups

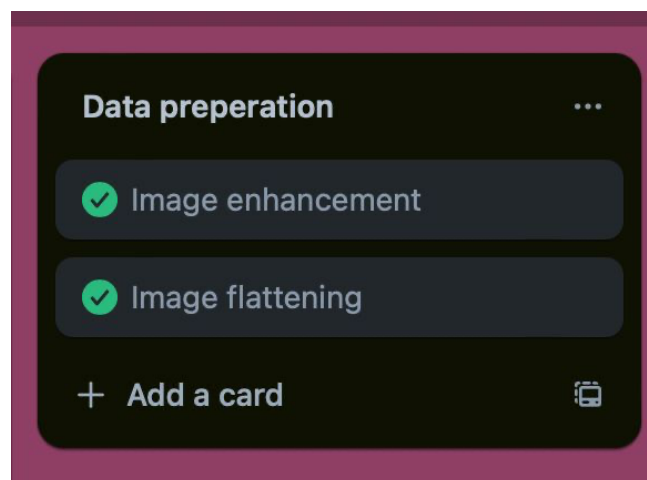
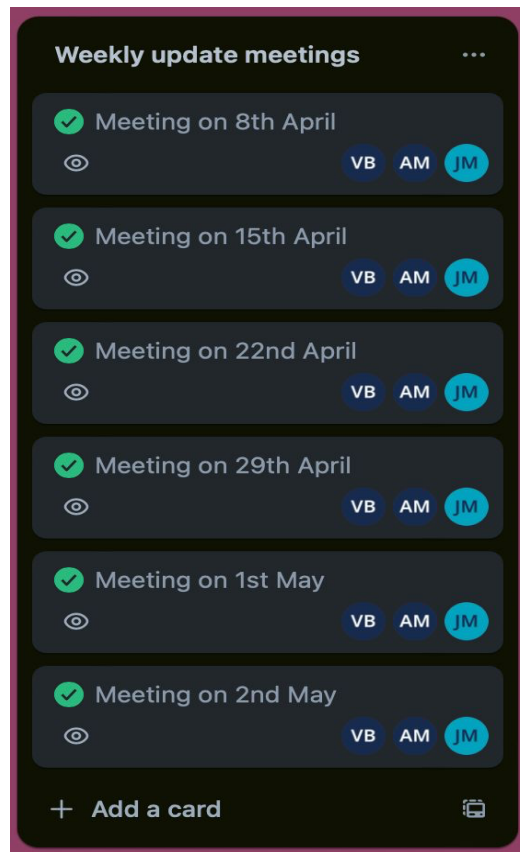
✓ Meet on 10th march

✓ Meet on 21st April

+ Add a card



Project Management Tool: Trello (Till Final)



Data preprocessing ...

✓ Image enhancement

✓ Document extraction

+ Add a card

Building DL Models ...

✓ Document Extraction

✓ Image enhancement

+ Add a card

Training Pipeline ...

✓ Document extraction

✓ Image flattening using cv2

✓ Image enhancement

+ Add a card

Evaluation and Visualisation

...

- ✓ Compare the ground truth with the predicted output

+ Add a card



Integrating the pipeline

...

- ✓ Built the end to end pipeline

+ Add a card



Testing and debugging

...

- ✓ Model debugging and correction

+ Add a card



Model Deployment

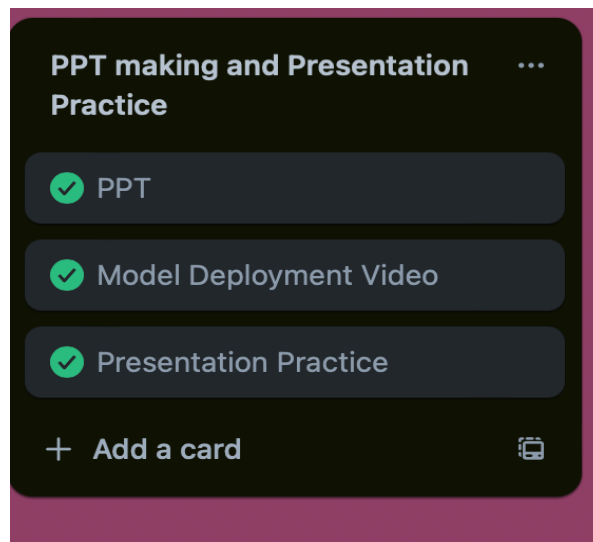
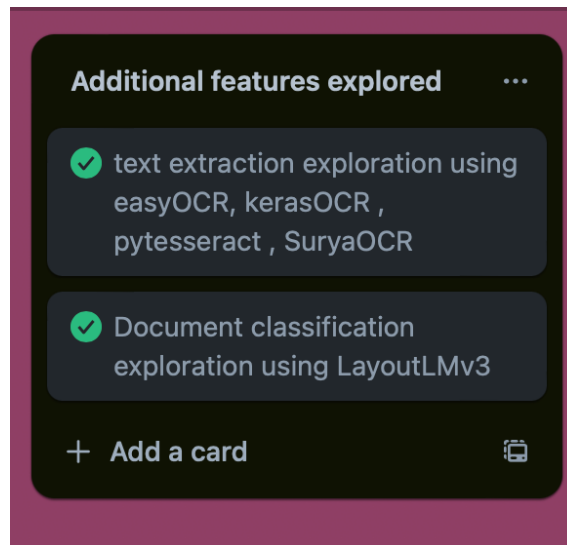
...

- ✓ FastAPI



+ Add a card





Link to Dashboard: <https://trello.com/b/zf5jCVee>

Comparison of Scanned Results with Benchmark (CamScanner)

In this project, we compare the results from our model with the outputs of CamScanner, which we are using as the benchmark. The original images contain various types of documents placed on different backgrounds, including clutter, dull images and poor lighting.

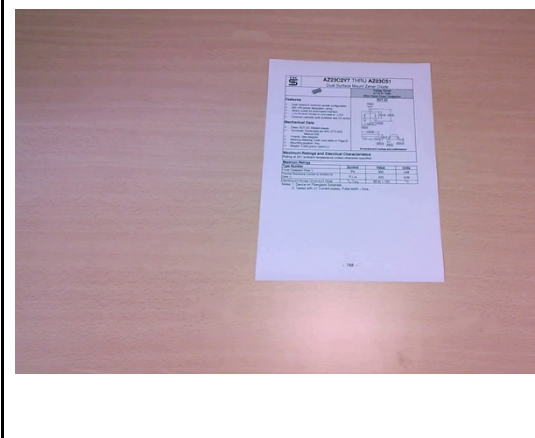
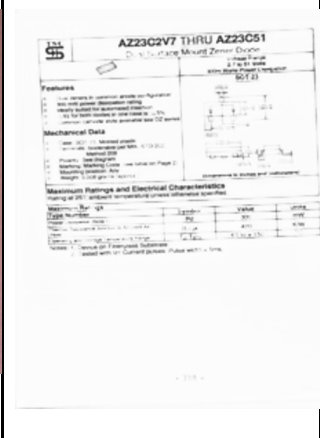
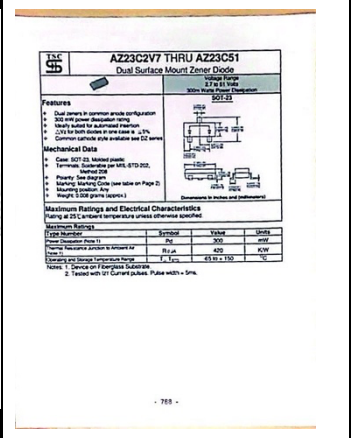
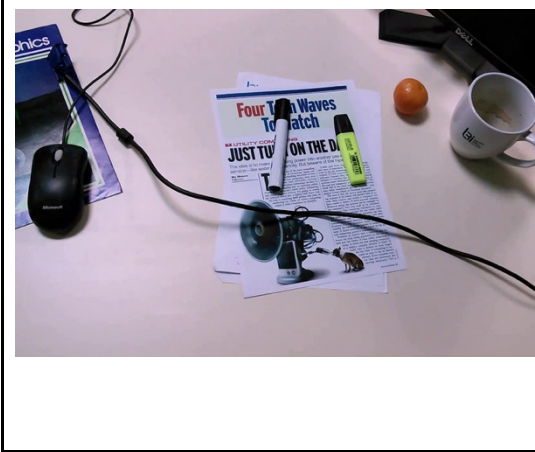
Our model performs quite well in extracting the document from the background. In these examples, especially in 1st, 3rd and 4th images, the scanned outputs are geometrically correct and look aligned, like the benchmark scanned documents.

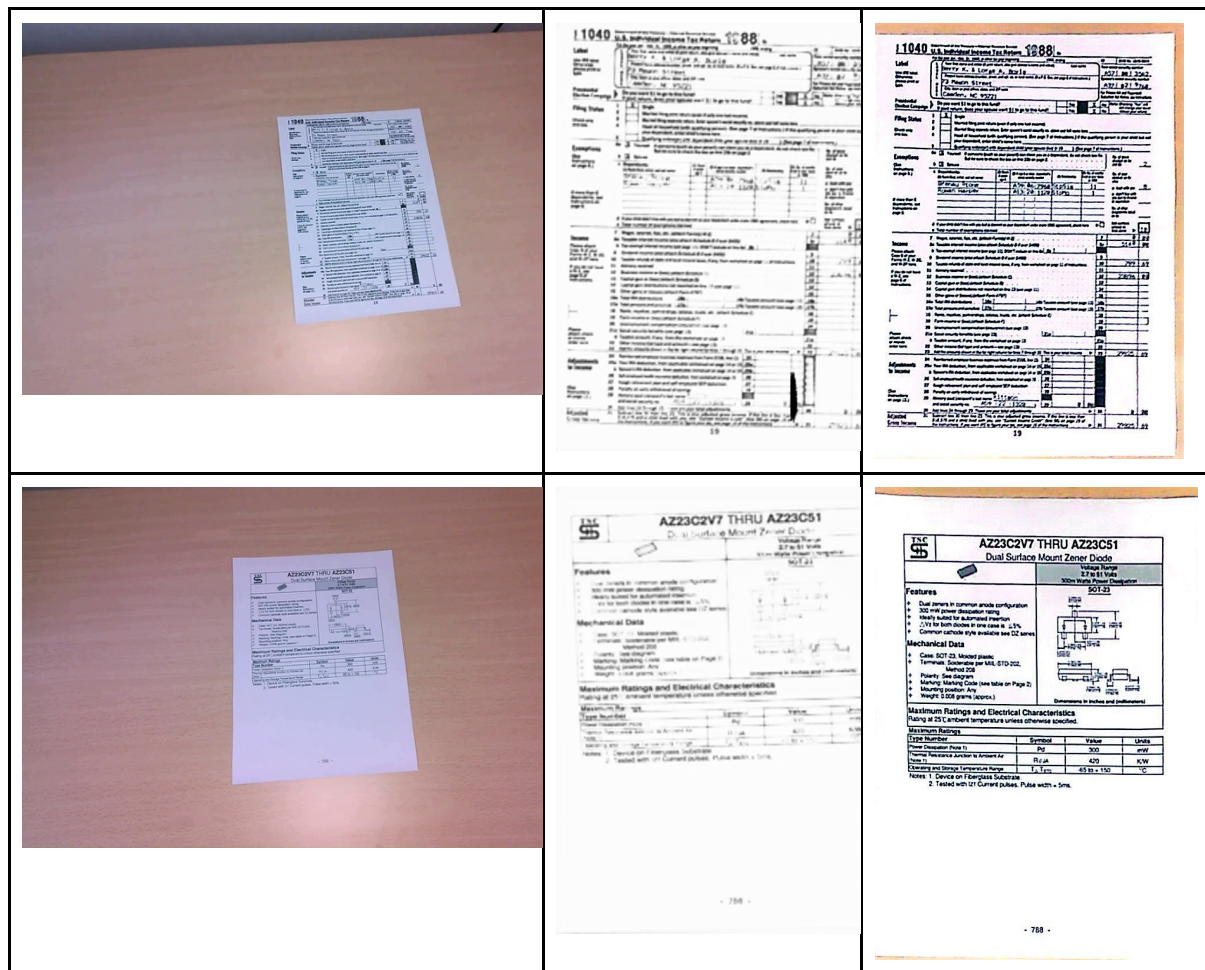
We could also notice that in the second row, despite the presence of noise like wires, pens, and other objects, our model successfully captured the entire document. In contrast, the CamScanner result missed a portion of the page, showing only half the document. This highlights our model's ability to handle complex scenes better in some situations.

However, there are still areas where our model needs improvement. One major issue is text clarity. While the main headings are visible, the rest of the text often appears overexposed. We could notice that Rows 3 and 4 especially show this problem, the document content becomes hard to read. On the other hand, CamScanner outputs a sharper image, with high contrast and a bit more readable text. Overall, our model shows good performance in extracting and aligning documents and even outperforms CamScanner in some cluttered scenarios. But it still needs enhancements in contrast, text clarity, and background cleaning to reach the quality of professional tools like CamScanner. With some refinements, especially in post-processing, our model has strong potential for real-world document scanning applications.

Qualitative Analysis: Visual assessment of document classification and text accuracy.

Quantitative Metrics: Classification Accuracy (Top-1, Top-5 Accuracy) for document categories , Precision, Recall, and F1-score for text-based classification.

Original Image	Scanned Images	Benchmark Images
		
		



Dataset :

<https://archive.ics.uci.edu/ml/datasets/NoisyOffice>

<https://www.kaggle.com/datasets/octaviusgaster/smartdoc2015-extracted-frames/data>

References :

- <https://opencv.org/>
- <https://learnopencv.com/automatic-document-scanner-using-opencv/>
- <https://onyekaokonji.medium.com/making-your-own-document-scanner-in-40-lines-of-code-1db3a360e3b7>
- <https://www.kaggle.com/datasets/octaviusgaster/smartdoc2015-extracted-frames>
- <https://www.kaggle.com/datasets/drcrabkg/financial-statements-clustering>
- https://docs.opencv.org/4.x/d9/dab/tutorial_homography.html
- <https://cs231n.github.io/convolutional-networks/>
- https://openaccess.thecvf.com/content_cvpr_2018/papers/Ma_DocUNet_Document_Image_CVPR_2018_paper.pdf
- <https://code.visualstudio.com/docs/python/tutorial-fastapi>
- <https://www.geeksforgeeks.org/fastapi-uvicorn/>
- <https://cs231n.github.io/convolutional-networks/>

Contribution Table:

<u>Team Member</u>	<u>Project Part</u>	<u>Contribution %</u>
<u>Arin Singh Mashta</u>	Image flattening, enhancement, DL models, model deployment, report	Equal contribution
<u>Jaya Meena</u>	Image flattening, enhancement, DL models, model deployment, report	Equal contribution
<u>Vishesh Bhargava</u>	Image flattening, enhancement, DL models, model deployment, report	Equal contribution